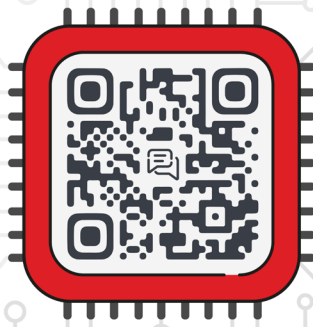




4



6



900 sp

نظري

كلية الهندسة المعلوماتية

السنة الأولى

أنواع التعليمات 3

د. مدحت الصوص

د. فراس ضعيف



برمجة 1

RB Informatcs; 14/12/2025

محتوى مجاني غير مخصص للبيع التجاري

تعرفنا في المحاضرات السابقة على (القراءة، الكتابة، الإسناد، الشرط) وفي هذه المحاضرة سنتعرف على تعليمة التكرار:

التكرار

1. التكرار while

تستخدم تعليمة طالما (while) لتكرار تنفيذ مجموعة من التعليمات الموجودة ضمن الحلقة طالما بقي الشرط محققاً. ويكون على الشكل:

الخوارزمية	لغة C++
طالما (الشرط) { مجموعة خطوات }	while (الشرط) { مجموعة الخطوات }

في حلقة التكرار من نوع while يتم فحص الشرط ثم تنفيذ التعليمات

ما الفرق بين الشرط (if) والتكرار (while)؟

الشرط	التكرار
يقوم ال compiler بفحص الشرط فإذا كان محققاً ينفذ التعليمات ويخرج إلى باقي الشرط	يقوم ال compiler بفحص الشرط فإذا كان محققاً ينفذ التعليمات ويعود لفحص الشرط مرة أخرى بعد أن يفرغ من تنفيذ التعليمات وعندما يختل الشرط يخرج من الحلقة ويكمل إلى باقي البرنامج

بعد المقارنة السابقة نلاحظ إنه إذا كان الشرط داخل الحلقة محققاً دوماً فسوف يتم تنفيذ التعليمات بشكل لا نهائي وبذلك نستنتج أن الشرط داخل (طالما) يجب أن يختل في لحظة من اللحظات لكي يتوقف تنفيذ التعليمات داخل حلقة الشرط، ويمكن أن نخل بالشرط عن طريق إضافة متحول ضمنه، وإضافة خطوة ضمن مجموعة الخطوات تقوم بتغيير هذا المتحول



مثال على ما سبق

شرط دون متغير	شرط مع متغير
طالما $(30 > 2)$ } اكتب "تعليمه ما" { هنا سيتم تنفيذ تعليمه الكتابة عدد لا نهائي من المرات	$x \leftarrow 1$ طالما $(x < 10)$ } اكتب "تعليمه ما" $x \leftarrow x + 1$ {

مثال: اكتب برنامج يقوم بكتابة عدد الأرقام من (1 إلى 100)

الخوارزمية	لغة C++
$x \leftarrow 1$ طالما $(x \leq 100)$ } اكتب x $x \leftarrow x + 1$ {	<pre>int x = 1; while (x <= 100) { cout << x << endl; x = x + 1; }</pre>

نقوم بإسناد قيمة ابتدائية لـ x قبل دخول الحلقة

ولا نقوم بإسناد قيمة ابتدائية لـ x داخل الحلقة؛ لكيلا تتم عملية الاسناد للقيمة نفسها في كل دخول للحلقة

- نضيف 1 إلى قيمة x بعد تعليمه الكتابة في حال كانت القيمة الابتدائية للمتحول x تساوي 1 وذلك من أجل طباعة القيمة 1 في أول دخول للحلقة ثم زيادتها إلى 2 وتخزينها في الذاكرة ليتم طباعته في الدخول الثاني للحلقة وهكذا
- وفي حال كانت القيمة الابتدائية للمتحول تساوي 0 فنقوم بإضافة 1 إلى قيمة x قبل تعليمه الكتابة وذلك من أجل زيادة قيمة المتحول من 0 إلى 1 في أول دخول ثم طباعته وهكذا

2. نفذ-طالما (do while)

تستخدم تعليمه نفذ-طالما (do while) لتكرار تنفيذ حلقة ما طالما بقي شرط الاستمرار محققاً وتكون على الشكل:

الخوارزمية	لغة C++
نفذ {مجموعة خطوات} طالما (شرط)	<pre>do {مجموعة خطوات} while (شرط);</pre> ملاحظة: نضع فاصلة منقوطة (;) بعد قوسي while في حلقة do-while لتمييزها عن تعليمه while العادية

في حلقة التكرار من نوع do-while يتم تنفيذ التعليمات ثم فحص الشرط

```
#include <iostream>
using namespace std;
int main () {
    int x = 1;
    do {
        cout << x << endl;
        x = x + 1;
    }
    while (x<=100);
}
```



ما الفرق بين حلقة while وحلقة do-while؟

While	Do-while
تقوم حلقة while بفحص الشرط ثم تنفذ التعليمات في حال كان شرط الاستمرار محققاً، بمعنى أنه إن لم يكن الشرط محققاً من أول فحص فلن يتم تنفيذ أي تعليمة	تقوم حلقة do-while بتنفيذ التعليمات بأول دخول ثم تقوم بفحص الشرط، أي أنها تنفذ التعليمات مرة واحدة على الأقل حتى إن لم يكن الشرط محققاً

3. من أجل (for)

تستخدم حلقة for لتكرار تنفيذ مجموعة من التعليمات بشكل مشابه لتعليمة while وتكون على الشكل:

```
(تعليمة 2 ; الشرط ; تعليمة 1) for
{
    مجموعة تعليمات
}
```

التعليمة 1: تنفذ عند أول دخول لحلقة for فقط ولا يتم تنفيذها إلا مرة واحدة فقط

التعليمة 2: تنفذ بعد كل دخول لحلقة for (بعد الانتهاء من تنفيذ مجموعة التعليمات داخل الحلقة)

مثال: كتابة الأعداد من 1 إلى 100 باستخدام حلقة for

```
for (int i = 1; i <= 100; i++)
{
    cout << i << endl;
}
```

ما الفرق بين حلقة while وحلقة do-while؟

تختلف حلقة for عن حلقة while بإمكانيتها في تنفيذ تعليمة أو مجموعة تعليمات ضمن (تعليمة 1) قبل الدخول إلى الحلقة، أي قبل فحص الشرط وقبل أي تنفيذ للتعليمات، وكذلك بإمكانية تنفيذ تعليمة أو مجموعة تعليمات ضمن (تعليمة 2) بعد كل دخول للحلقة أي بعد كل تنفيذ لمجموعة التعليمات

- يمكن كتابة أكثر من تعليمة واحدة ضمن جزء (تعليمة 1) والجزء (تعليمة 2) وعندها يجب أن توضع فاصلة عادية بين التعليمات ضمن الجزء الواحد مثل
(تعليمة,تعليمة ; الشرط ; تعليمة ,تعليمة) for
- تعتبر كل من التعليمة 1 والتعليمة 2 والشرط أجزاءً اختيارية ضمن تعليمة for فيمكن عدم كتابة أي منها ولكن يجب دائماً كتابة الفاصلتين المنقوطين ضمن قوسي التعليمة ونميز الحالات الآتية
1- إذا حذفنا التعليمة 1 يجب أن نصرح عنها خارج الحلقة
2- إذا حذفنا التعليمة 2 أو الشرط ستصبح الحلقة لا نهائية
- متى نستخدم while ومتى نستخدم for
يمكننا استخدام while و for لجميع الحالات ولكن إن كان لدينا

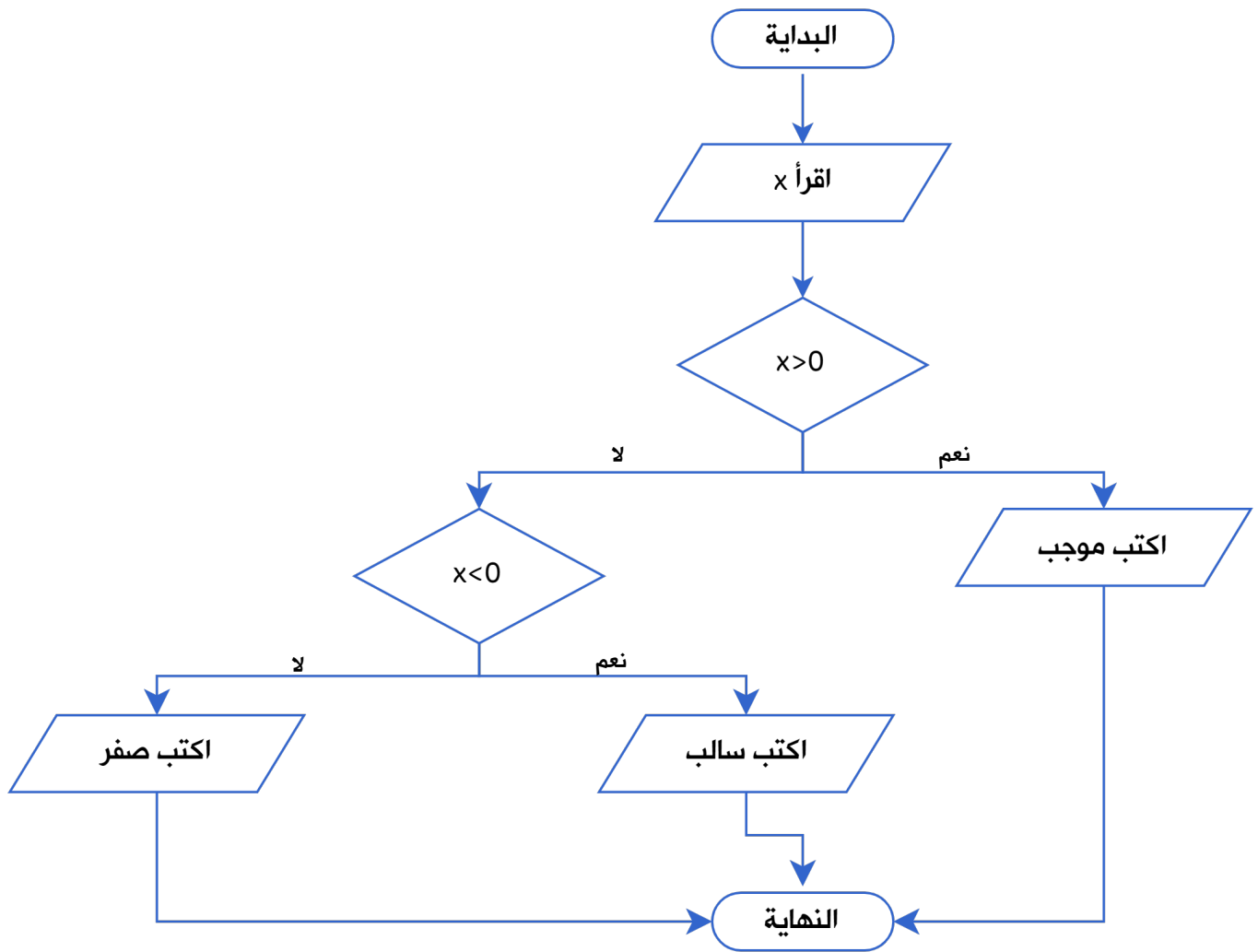
تكرار حتى يختل شرط ما	تكرار لخطوات عدد من المرات
الأفضل استخدام while	الأفضل استخدام for

إلى هنا نكون قد أنهينا اللغة النصية للخوارزمية

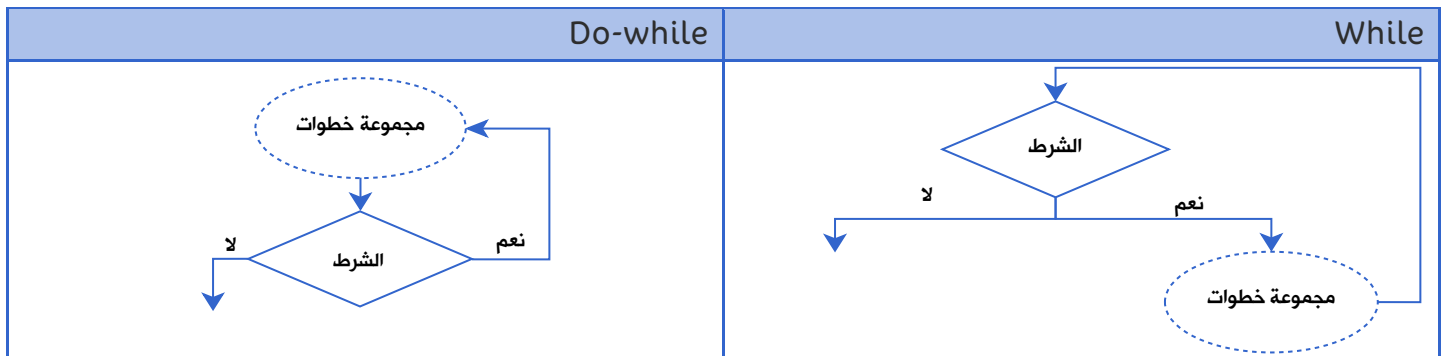
اللغة البيانية

اللغة البيانية: نستخدم لتسهيل فهم أو تقريب معنى الحلقات واللغة إلينا

التعليمة	لغة نصية	لغة رسومية
بداية أو نهاية الخوارزمية	البداية / النهاية	
القراءة والكتابة	اقرأ / اكتب	
الإسناد	←	
الشرط	إذا	
التكرار	طالما	



الفرق بين while و do-while باللغة البيانية



تعليلة continue

توضع هذه التعليلة حصراً ضمن الحلقات وتسمح بإنهاء تنفيذ التكرار الحالي من الحلقة والقفز مباشرة لفحص شرط استمرار الحلقة للدخول في تكرار جديد عند تحققه. أي أن جميع التعليلات الواردة بعد تعليلة continue ضمن الحلقة لا يتم تنفيذها، ويتم القفز مباشرة لاختبار شرط الحلقة
مثال: لو أردنا طباعة جميع الأعداد من 1 إلى 100 باستثناء العدد 50

```
#include <iostream>
using namespace std;
int main () {
    for (int x = 1; x <= 100; x++)
    {
        if (x == 50)
            continue;
        cout << x << endl;
    }
    return 0;
}
```



هنا من أجل كل دخول للحلقة يتحقق فيه الشرط $x == 50$ يتم تنفيذ تعليلة continue أي يتم تجاوز تنفيذ تعليلة الطباعة cout

تعليلة break

توضع هذه التعليلة ضمن الحلقات أو ضمن تعليلة switch تقوم هذه التعليلة بالخروج من الحلقة (أو من تعليلة switch) وتنقل التنفيذ إلى التعليلة التالية مباشرة للحلقة (أو لتعليلة switch) تعتبر هذه التعليلة الحل الوحيد للخروج من الحلقات غير المنتهية التي يبقى شرطها محققاً دائماً
مثال لو أردنا طباعة جميع الأعداد من 1 إلى 100 باستخدام حلقة غير منتهية مع استخدام break للخروج نكتب:

```
#include <iostream>
using namespace std;
int main () {
    int x = 1;
    while (true) {
        if (x == 101)
            break;
        cout << x << endl;
        x++;
    }
    return 0;
}
```

